

Orbital Waddle

Orbital Waddle (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): You play an astronaut penguin whose partner is kidnapped by the Astrorca and taken to Planet 67. Journeying across a chain of alien planets — **each a rotating isometric cube world** — you clear timed enemy waves to earn the key and locate a hidden **Thingamajig** to power the rocket onward. Combat is hack-and-slash roguelite (in the vein of Hades / Megabonk): start with a snowball launcher, grow an arsenal of weapons, upgrades, abilities and stat boosts. Each planet has its own terrain, enemy mix and theme, and **walking off an edge smoothly rotates the whole world so a new cube face becomes the ground**. Named technical features: (1) a **planet-rotation traversal engine** that reorients terrain + the gravity vector as the player crosses an edge while keeping every active enemy/projectile/object stable in the local frame; (2) an **ECS sustaining 100 concurrent enemies per wave** without frame drops on Android; (3) a Thingamajig relocation system that updates a HUD compass; (4) a **data-driven weapon/ability/stat economy** backed by a **Firebase** layer for cloud-save sync, server-side achievement unlocks and a server-validated global leaderboard. Loop: the game loop, ECS, planet rotation and enemy AI run in one fixed-timestep C++ loop (Android Studio + C++ via JNI) targeting 100 live enemies; adding an enemy/wave is a data change. Tools: a **planet & wave editor** (terrain, spawn zones, Thingamajig points), a **data-driven config tool** (weapon stats, upgrade trees, enemy params, post-rescue difficulty tiers), and a **CI pipeline**. Design: the rotating planet world; a 5-enemy roster (Astrorca, RoboLi Seals, Space Petrels, F.O.X, Polaroids) with distinct movement/attack patterns; a wave→key + Thingamajig loop across 4 planets (3 themed + Planet 67 finale); and a **New Game+ completionist loop** with remixed waves and **cosmetic-only** rewards (no pay-to-win). Platform: **Mobile, native Android**.

References:

- [Hades on Steam](#) (the hack-and-slash roguelite the proposal names: escape-run structure, a growing arsenal of weapons and stackable Boons/upgrades, thousands of build combinations, isometric combat — the roguelite arsenal + New Game+ precedent)
- [Fez — Wikipedia](#) (a world you traverse by **rotating between faces**, so a different plane becomes navigable ground — the closest published precedent for the cube-face rotation-traversal mechanic)
- [Super Mario Galaxy — Wikipedia](#) (small self-contained planetoids with all-around gravity that let you walk around a body as its gravity reorients — the planet-gravity-reorientation precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Orbital Waddle
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	●
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — the stated platform, Android Studio + C++ via JNI; device: Pixel 9a) — a touch roguelite with an on-screen HUD compass; the Native PC build is the development/tools vehicle (the planet/wave editor + config tool). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for cloud-save sync, server-side achievement unlocks and the server-validated global leaderboard.

Front end: An isometric renderer of the cube-world planet (terrain, the penguin, up to ~100 enemies, projectiles and VFX) with the ****smooth edge-crossing world rotation****, the wave/health/arsenal HUD and the Thingamajig compass. **Languages:** Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES drawn in a GLSurfaceView for the realtime isometric world (the rotation is a 3D transform of the camera/world); Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: One fixed-timestep C++ loop running the ECS (≥ 100 enemies/wave), the planet-rotation traversal (terrain + gravity-vector reorientation, keeping entities stable in the local frame), enemy AI, the wave→key→Thingamajig progression and the weapon/ability/stat economy + a Firebase application-communication layer. **Languages:** C++ (engine via JNI); Firebase-backed services on the Extended tier.

Backend scope: Local now, server-ready (T2) — Firebase cloud-save / achievements / leaderboard seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is heavily load-bearing — one fixed-timestep loop drives the ECS at a target of **100 concurrent enemies/wave on Android**, the planet-rotation reorientation (terrain + gravity vector, entities stable in the local frame), enemy AI, waves and the arsenal economy.
- *Data-driven:* the ECS separates data from behaviour — enemy types, wave patterns, weapon/upgrade/stat trees, per-planet terrain/theme and the post-rescue difficulty tiers are all data, so a new enemy or wave is a data change, authored via the editor/config tools.
- *2D:* isometric sprite/tile rendering of the cube-face world, enemies, projectiles, VFX and the HUD compass (custom-rendered). *3D:* ● the edge-crossing rotation and gravity reorientation are a 3D camera/world transform layer over the isometric art — a strong-students concern, not a full 3D pipeline. *Networking:* no real-time multiplayer; the Firebase cloud-save / achievement / leaderboard calls are async client↔service (Application Communication ✓), so a real-time Networking System is optional (●).
- *Custom tool:* a **planet & wave editor** (terrain, enemy spawn zones, Thingamajig relocation points per planet) **and a data-driven config tool** (weapon stats, upgrade trees, enemy params, difficulty tiers) — rebalance without recompiling.

- *External DB*: Firebase cloud saves, achievements and the global leaderboard. *Auth* »: accounts for cloud save. *Analytics* »: optional — wave/death/build telemetry for balancing.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + one fixed-timestep loop (C++17/20) driving the ECS, planet rotation, enemy AI and input; CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; M1G5 rotation/physics and M1G8 enemy AI run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / ECS**

Tech & dependencies

- **Tech:** A cache-friendly ECS in C++ (data separated from behaviour) built to sustain ~100 enemies/wave, with enemy types, wave patterns, weapon/upgrade/stat trees, per-planet terrain and difficulty tiers all defined in JSON and hot-reloadable — a new enemy/wave is a data change.
- **Prerequisites:** G1 (loop & system update order).
- **Feeds into:** M2G2 (the editor/config tools edit this data), M1G8 (enemy data), M2G13 (economy).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui debug overlays (ECS counts, rotation state, gravity vector), Tracy profiling to sustain **100 enemies/wave on Android without frame drops**, Catch2 tests for the loop/rotation/data loading, and a CI pipeline (GitHub Actions Windows+Android matrix) building and testing every commit.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the 100-enemy budget M2G1 validates on-device.
- **G4 2D Graphics** — the isometric cube-world renderer (terrain, penguin, enemies, projectiles, HUD compass) is the whole read of the game.
Tech & dependencies
 - **Tech: Web build:** Isometric sprite/tile renderer with a camera, batching for ~100 enemies + projectiles, depth sorting on the cube face, and the HUD/compass; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** G1 (loop & window).
 - **Feeds into:** M2G4 (advanced combat/rotation VFX); every visible frame.
- **G5 Physics & Collision** — the **planet-rotation traversal**: reorient terrain + the gravity vector as the player crosses an edge, keeping every enemy/projectile stable in the local frame, plus combat/knockback collision.
Tech & dependencies
 - **Tech: Web build:** C++ + GLM math for the gravity-vector reorientation and the smooth face-to-face rotation transform, local-frame stability for active entities during the turn, and collision/knockback for combat; broadphase (grid) for ~100 enemies. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** G1 (tick), G2 (terrain/entity data).
 - **Feeds into:** The signature traversal; M2G1 1-class polish; combat feel.
- **G8 Enemy AI & Pathfinding** — the 5-enemy roster (Astrorca, RoboLi Seals, Space Petrels, F.O.X, Polaroids), each with a distinct movement/attack pattern.
Tech & dependencies
 - **Tech:** Finite-state machines + steering/pathfinding on the cube face for the 5 enemy types, tuned per type from data, scaling to ~100 concurrent agents (custom C++).

- **Prerequisites:** G1 (tick), G2 (enemy data), G5 (movement on the rotating terrain).
- **Feeds into:** M2G10 (advanced AI + the Astrorca boss + difficulty tiers); the timed-wave loop.

M2:

• G1 ★ Architecture — Optimization & Platform Validation

Tech & dependencies

- **Tech: Web build:** Tracy profiling to sustain 100 enemies/wave through a rotation, ASan on MSVC, ECS memory pools, and validation on-device (Pixel 9a, Native Android); the Firebase/Linux-server seam (cloud save + achievements + leaderboard) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable and a stable 100-enemy budget across a rotation.

• G2 ★ Content Editor — Core (Planet & Wave Editor) — place terrain, enemy spawn zones and Thingamajig relocation points per planet; the config tool for weapon/upgrade/enemy/difficulty-tier tuning.

Tech & dependencies

- **Tech:** Dear ImGui editor authoring per-planet terrain, spawn zones, Thingamajig points and wave patterns as JSON, plus a config panel for weapon/upgrade/enemy/difficulty-tier values, with round-trip serialization, live preview and undo/redo.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (live preview of the cube face).
- **Prerequisites:** G2 (data model), M1G4 (rendering for preview).
- **Feeds into:** Fast content/balance iteration; the custom-tools showcase.

• G3 ★ CI — CMake, Code Quality, Build Standards & Packaging

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android Debug builds, automated tests (simulation loop, rotation, data loading) and packaging via GitHub

Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

- **G8 Firebase — Accounts, Leaderboards & Analytics** — cloud-save sync, server-side achievement unlocks and the server-validated global leaderboard.

Tech & dependencies

- **Tech:** Firebase Auth accounts, Firestore for cloud saves + achievements + the leaderboard (server-validated to prevent forged scores), optional Analytics, via the REST API from C++.
- **Prerequisites:** G2 (save/economy data model).
- **Feeds into:** Cross-device saves, the completionist achievements, and competitive leaderboards.

- **G10 AI — Advanced** — the Astorca boss (Planet 67 finale) and the New Game+ remixed/harder enemy tiers.

Tech & dependencies

- **Tech:** Behaviour trees for the Astorca boss (phases/patterns), difficulty-tier scaling of the 5-enemy roster and remixed NG+ waves, all data-driven (custom C++).
- **Prerequisites:** M1G8 (enemy AI foundation).
- **Feeds into:** The finale + the New Game+ completionist loop.

- **G13 Gameplay Systems II** — the **weapon / ability / upgrade / stat economy** and the New Game+ completionist loop (cosmetic-only rewards).

Tech & dependencies

- **Tech:** Composable C++ systems over the ECS for weapons, abilities, upgrade trees and stat boosts (start: snowball launcher), plus the New Game+ remix + cosmetic unlocks (skins, launcher recolours — no pay-to-win), all balancing-data-driven.
- **Prerequisites:** M1G2 (ECS/economy data), M2G2 (config tool authors it).
- **Feeds into:** The roguelite depth and the replay loop at showcase; T2 arsenal expansion.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 2 Art (9) — four distinct planet themes, a 5-enemy roster + the Astrorca boss, the penguin, the growing arsenal and its VFX, and the cosmetic skins are a large art surface, so two artists carry it; a single designer owns the roguelite economy (weapons/upgrades/stats), the wave/planet/progression design and the New Game+ tiers (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the economy + wave/tier design outgrows one designer, trade the 2nd artist for a 2nd designer.

Scope note (advisory): a novel planet-rotation engine + a 100-enemy ECS + 5 enemies + a boss + 4 planets + an arsenal economy + Firebase + two editors is a very large surface — treat the rotation traversal on one planet, the ECS + ~2–3 enemies, the wave→key→Thingamajig loop and the planet/wave editor as the M1/M2 floor, with the full roster, 4 planets, the arsenal economy and New Game+ as the stretch. The **rotation-traversal engine is the highest technical risk** — prototype it first.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the rescue premise, the rotating-planet traversal, the wave→key→Thingamajig loop and the roguelite arsenal; primary benchmark e.g. Hades, with a Singapore benchmark for the local-awareness criterion)
- M1D2 Core Gameplay Loop Prototype (a greybox proving the ****edge-crossing rotation + hack-and-slash combat**** is fun and readable — the highest-risk mechanic) · prototype in Unity
- M1D6 Systems & Economy Design (the weapon/ability/upgrade/stat economy, the wave/key pacing and the cosmetic-only NG+ rewards — every system with a purpose)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (combat, arsenal, wave and difficulty-tier balance via the config tool, no code changes)

- **UI/UX M1** (graded by the Design instructor for the Games category):
 - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the wave/health/arsenal HUD and the ****Thingamajig compass****, legible over a busy 100-enemy screen and stable through a rotation.
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — HUD, arsenal/upgrade menus, compass and results navigable unaided on the Native-Android target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished HUD + upgrade-choice UX tested with players on the Pixel 9a.
- **Art M1:** 2 artists justified (4 planet themes + 5 enemies + boss + arsenal VFX)
 - M1A1 ★ Art Bible / Proof of Concept (the isometric cube-world direction — palette per planet, readability at 100-enemy density)
 - M1A2 Character Concepts & Model Sheets (the penguin, the 5 enemies and the Astorca boss)
 - M1A3 Environment, Props & VFX Concepts (the 4 planet themes/terrain + the arsenal/ability effect language)
 - M1A4 UI Art Concepts & Typography (HUD, Thingamajig compass, arsenal/upgrade screens)
 - M1A5 Animation Foundations (penguin, enemy and attack cycles; the rotation transition)
 - M1A6 Early Asset Production — Prototype Support
- **Art M2:**
 - M2A1 ★ Critical Asset Production & In-Engine Visual Quality
 - M2A2 Character Animation — Extensive (the roster, the boss, ability reactions)
 - M2A3 Environment Alive — Ambient Animation & Layering (the 4 living planet worlds + the rotation)
 - M2A4 UI Art — States & Feedback (upgrade-choice, wave-clear, compass states)
 - M2A5 VFX — Critical Effects & Transitions (weapons, abilities, the world-rotation transition — the signature moment)
 - M2A6 Cutscenes & Storyboards (the kidnapping intro / Planet 67 confrontation / rescue)

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (per-planet BGM, weapon/hit/ability, enemy and the rotation "whoosh" cues), suitable and correctly triggered.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for a 100-enemy combat mix.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (per-planet BGM + weapon, ability, enemy, boss, wave-clear and world-rotation SFX, ≥6 types) that stays legible under heavy combat.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; a distinct rotation-transition cue).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the planet & wave editor), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*